



UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

AI-Driven Bot Implementation for Process Automation: Towards Autonomous Control

University supervisor:

Prof. Alberto Trombetta

External supervisor:

Mirko Giordani (ALTEN)

Dissertation by:

Luca Volonterio

Matricola 744690

Bachelor thesis

July 2024

AI-Driven Bot Implementation for Process Automation

Towards Autonomous Control

Luca Volonterio

Abstract

This thesis presents the development and implementation of an AI-driven bot designed to automate processes within a service-oriented business environment.

The bot leverages the capabilities of Selenium WebDriver for web automation and the OpenAI GPT-3.5 model for decision making. Through the integration of these tools, the goal is for the software to be able to autonomously manage and process IT service requests.

The project is divided into several stages, beginning with the development of core action automation and error handling mechanisms to be able to implement RPA tasks.

Subsequent phases involved building an automatic request parsing system and integrating AI for determining how to solve each request when possible.

The results demonstrate a significant improvement in efficiency and service quality, with the bot potentially being able to handle approximately 70% of incoming tickets.

In conclusion, this thesis provides an early example of AI used for decision making inside a software program, ultimately underscoring the value of AI-driven automation in modern times.

Contents

1	Introduction	1
1.1	Context	1
1.2	Problem statement	2
1.3	Project scope	3
2	Tools	5
2.1	Selenium WebDriver	5
2.2	OpenAI API	6
2.2.1	Playground	6
3	Project Overview	9
3.1	Enviroment setup	9
3.1.1	Python	9
3.1.2	Dependencies	10
3.2	RPA Implementation	12
3.2.1	Actions	12
3.2.2	Error handling	17
3.2.3	Configuration	20
3.3	Ticket scraping	22
3.3.1	Functioning	22
3.3.2	Storage	24
3.3.3	State diagram	26
3.4	Request parsing	27
3.4.1	Querying GPT	27
3.4.2	Approving a solution	28
3.4.3	Executing	29
4	Results	31
4.1	Bot Performance	31
4.1.1	Execution time	31
4.1.2	Error rate	32

4.2	Qualitative Analysis	33
4.2.1	Request coverage	33
4.2.2	Time saved	34
5	Conclusions	35
5.1	Key points	35
5.2	Limitations	36
5.3	Future work and final remarks	37

List of Figures

1.1	Helix ITSM architecture (from docs) [1]	2
1.2	Common RPA use cases (from article) [2]	3
2.1	Selenium WebDriver architecture (from article) [3]	6
3.1	State diagram for the <i>Ticket</i> class	26
3.2	Legend for State transitions	26
3.3	Legend for state names	27
4.1	Comparison of Execution Times for Three Tasks: Human vs. Bot	32
4.2	Add	33
4.2	Link	33
4.2	Edit	33
4.2	Rates of Successes, Errors, and Warnings	33
4.3	Ticket categories	34
5.1	Example of multi agent system (from docs) [4]	38

Listings

3.1	Creating VENV	9
3.2	Installing requirements	10
3.3	Code snippet for opening a web page	10
3.4	Code snippet for initializing openAI client object.	10
3.5	Code snippet for making a request to the API.	11
3.6	Sample JSON response from the API.	11
3.7	Function definition for filling input	12
3.8	Function definition for clicking a button	13
3.9	Function definition for selecting an option from a dropdown list	14
3.10	Function definition for switching to another tab.	16
3.11	Function definition for switching to and from a nested iframe.	17
3.12	Function definition for initializing a Logger object.	18
3.13	Logger usage in the application.	19
3.14	Function definition for handling a simple error popup.	19
3.15	Example of the config.yml file.	20
3.16	Function definition and example of reading the configuration file.	21
3.17	Class definition for a table iterator.	22
3.18	Code snippet for iterating on table.	23
3.19	Functions definitions to dump and load Ticket objects.	24
3.20	Functions definitions for saving CSV file.	25
3.21	Function definition for querying GPT model.	27
3.22	Function definition for displaying and handling approval prompt.	28
3.23	Code snippet for executing ticket solution.	29

1

Introduction

In an extremely digitalized and interconnected world, the finance sector produces lots of transactions that need to be processed, memorized, and secured on dedicated infrastructure that is not normally owned and managed by banks directly; instead, this aspect is often delegated to specialized companies.

1.1 Context

The involved company is an important service-oriented organization specialized in banks and financial institutions, that provides outsourcing solutions regarding infrastructure and facility management. Its aim is to take on the provisioning and governance of hardware to allow banks to transparently deploy, scale and easily use the software they need while providing impeccable security and partitioning, granted by granular access control hosted on ultramodern systems.

To maximize efficiency while keeping the organizational overhead to the minimum, management of the IT infrastructure is carried out according to guidelines provided by the ITIL [5] framework, using tools like BMC Helix ITSM [6] that are meant to be comprehensive toolboxes for IT teams.

What these tools offer includes help desk, asset and change management, automation for streamlining processes and data analytics and insights useful in spotting bottlenecks or critical points in business operativity. In this context, ALTEN [7] handles the actual requests, incidents and problems being reported inside the ITSM platform that range

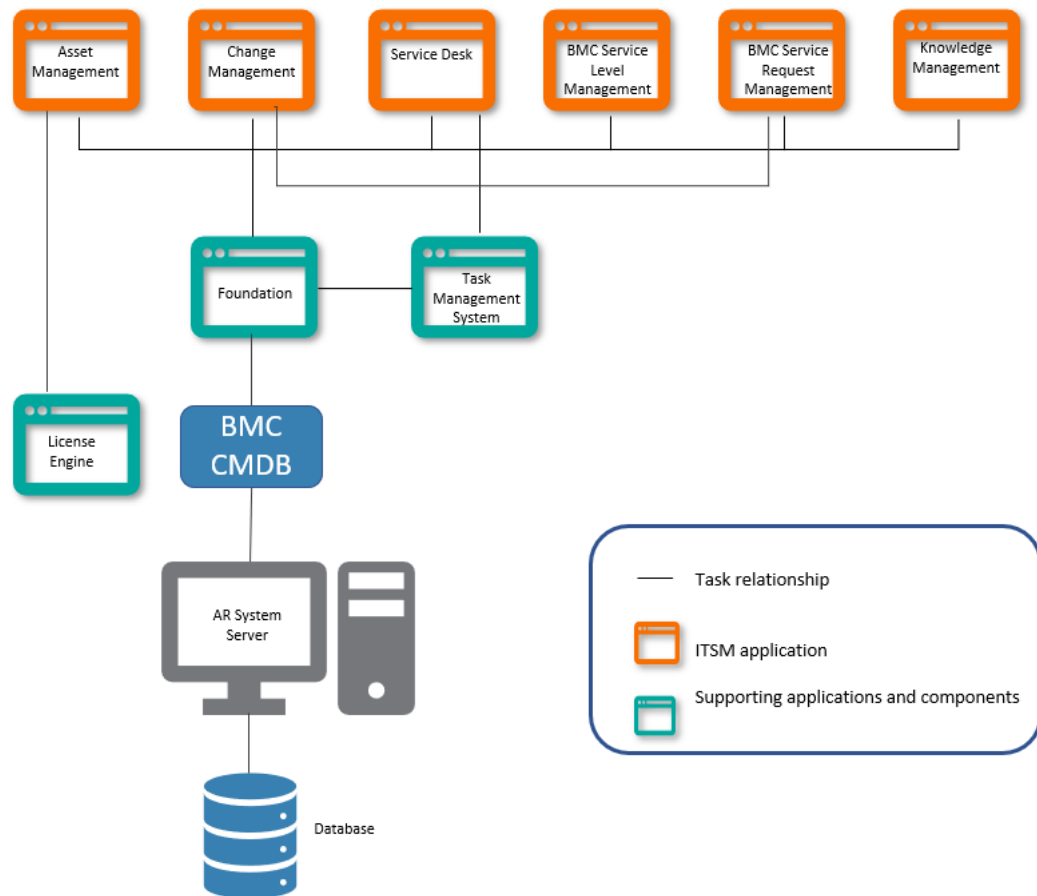


Figure 1.1: Helix ITSM architecture (from docs) [1]

from activating users and managing their identity to generating reports and dealing with bugs and config issues of the platform itself.

1.2 Problem statement

Some of the challenges that an ALTEN employee in this role faces daily can be reduced to adding objects into the system, managing groups and licenses linked to them and dismissing them. Clearly, these specific kinds of tasks are repetitive and can be easily and conveniently automated, especially when dealing with a considerable number of tasks that can pile up and cause great cost overall.

So, this thesis explores the idea of a bot that's able to carry out this process automatically (and hopefully *autonomously* in the future) while keeping a humans like mode of operation, that is working on a front-end rather than directly on the data

thus circumscribing the range of action of the automata and minimizing the risk of tampering the system's data.

1.3 Project scope

This software was initially just developed to process data in bulk in the context of a massive migration just like any RPA[8] (Robotic Process Automation) tool, such as UiPath[9] would; however, the scope changed after it became clear that it could as well be used in different settings.

This first stage lasted about a month, and most of the effort at this point was taken up by scripting, reading docs and designing software components to ultimately feed huge CSVs into the system, finally saving a lot of time.

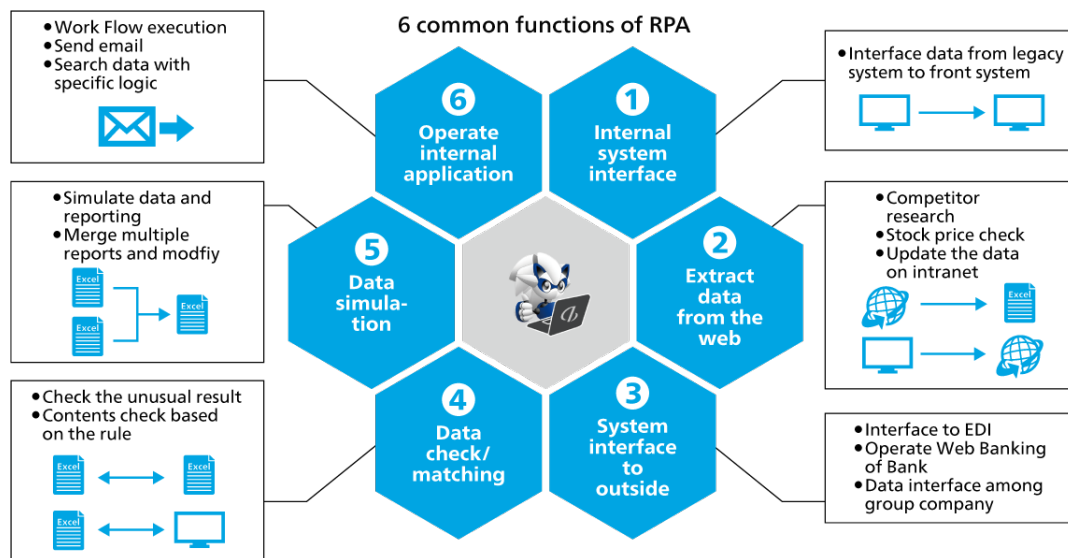


Figure 1.2: Common RPA use cases (from article) [2]

Subsequently however, as the system was used to process daily requests, it just needed basic input configuration and the focus shifted towards perfecting it for it to be better usable and resilient to errors. Here came the idea of studying the feasibility of a second part that could orchestrate and configure the bot itself, with minimum human intervention.

This was made by building a scraper capable of navigating the corporate ticketing system and parsing all the significant requests into serialized objects, to be stored and processed and additionally analyzed to gain insights on the most common kinds of tasks that needed to be performed.

Finally, these objects would be fed into a parser that, with the help of OpenAI's GPT

API, could try to elaborate a solution to each problem using the available tools, when possible.

2

Tools

To develop this project, two main tools have been used and here an overview of their architecture and functioning is provided. The Python language has been exclusively used for its simplicity in reading and writing it, and thanks to the bindings provided for both following software pieces.

Please note that most of the information here directly quotes documentation that can be found on the official websites as cited in the bibliography.

2.1 Selenium WebDriver

From Project's GitHub [10]: *Selenium is an umbrella project encapsulating a variety of tools and libraries enabling web browser automation. Selenium specifically provides an infrastructure for the W3C WebDriver specification [11] — a platform and language-neutral coding interface compatible with all major web browsers.*

Specifically, from Docs [12]: *WebDriver drives a browser natively, as a user would, either locally or on a remote machine using the Selenium server, marks a leap forward in terms of browser automation. Selenium WebDriver refers to both the language bindings and the implementations of the individual browser controlling code. This is commonly referred to as just WebDriver.*

So, Selenium Webdriver is a robust and well-known framework for automating web browser interactions that is mainly known for its cross-browser testing capabilities, but in this case was used as a low level automation tool.



Figure 2.1: Selenium WebDriver architecture (from article) [3]

2.2 OpenAI API

From Wikipedia [13]: *OpenAI is an American artificial intelligence (AI) research organization founded in December 2015, researching artificial intelligence with the goal of developing "safe and beneficial" artificial general intelligence, which it defines as "highly autonomous systems that outperform humans at most economically valuable work."*

In 2018, the organization released the first foundational GPT model, that is a LLM (Large Language Model) based on the transformer architecture. GPT stands for Generative Pre Trained, which means that the model is pre-trained on large data sets of unlabelled text and is capable of producing human-like content. These technologies developed exponentially in the last few years and more and more people and companies started using them.

From docs [14]: *the API today provides a general-purpose "text in, text out" interface, allowing users to try it on virtually any English language task. Given any text prompt, the API will return a text completion, attempting to match the pattern you gave it. You can "program" it by showing it just a few examples of what you'd like it to do; its success generally varies depending on how complex the task is.*

2.2.1 Playground

OpenAI also provides a Playground, i.e. a platform mainly intended for developers to experiment with different models and tweak their parameters according to a specific use case. The Playground facilitates an intuitive understanding of how different configurations impact model performance, enabling users to iteratively refine their approach to achieve optimal results. By offering features such as real-time feedback, parameter sliders, and

output visualizations, the platform supports a hands-on, experimental approach to AI development.

3

Project Overview

This section focuses on the actual implementation of this thesis; first, a setup guideline is provided and then some technical details are discussed. Please note that due to industrial secrets, some information is purposefully left out.

3.1 Enviroment setup

3.1.1 Python

First, Python needs to be installed and added to the PATH environmental variable by using the official installer from Python website [15].

Once installed, best practices suggest creating a virtual environment in the project's directory and activating it with the following bash commands 3.1.

```
1 cd </project/path>
2 python -m venv .
3
4 source ./bin/activate # UNIX
5 .\scripts\activate.bat # WINDOWS
```

Listing 3.1: Creating VENV

3.1.2 Dependencies

The project's dependencies are listed in the *requirements.txt* file. To install them, it is sufficient to run the following command 3.2.

```
1 pip install -r requirements.txt
```

Listing 3.2: Installing requirements

An overview of the main dependencies is hereby provided.

Selenium

Selenium Python bindings provide a simple API through which all functionalities of Selenium WebDriver can be accessed in an intuitive way. To do this, the browser's driver is needed, and this will be downloaded automatically with the installation of the selenium pip package by Selenium Manager, a command-line tool written in Rust that provides automated driver and browser management for Selenium.

After installing the pip package, controlling one of the installed browsers is possible with the following syntax 3.3.

```
1 from selenium import webdriver
2 driver = webdriver.Chrome()
3 driver.get("http://www.python.org")
```

Listing 3.3: Code snippet for opening a web page

In all following snippets, please assume this value for the variable *driver*.

OpenAI

The OpenAI API also comes in a pip package [16], and it enables the developer to query a GPT model of choice using a personal API key. Please note that in the following snippet 3.4 the environment variable holds the value of the secret API key, so that its sharing across multiple developers can be avoided.

```
1 import os
2 from openai import OpenAI
3 client = OpenAI(
4     api_key=os.environ.get("OPENAI_API_KEY"),
5 )
```

Listing 3.4: Code snippet for initializing openAI client object.

Inside the request 3.5, developers can specify which model to use and provide an array of messages for the model to reply to. These messages can be either system or user ones: the former specifies how the model should behave in general while the latter gives instructions about a specific task. Other parameters can be specified such as the temperature (the

higher it is, the more "randomness" the answer has) and the max number of tokens (to prevent excessive costs).

```
1 response = client.chat.completions.create(  
2     model="gpt-3.5-turbo",  
3     messages=[  
4         {  
5             "role": "system",  
6             "content": "You will be provided with statements, and your task is  
7             to convert them to standard English."  
8         },  
9         {  
10            "role": "user",  
11            "content": "She no went to the market."  
12        }  
13    ],  
14    temperature=0.7,  
15    max_tokens=64  
16 )
```

Listing 3.5: Code snippet for making a request to the API.

The response 3.6 has one or more answers (this behavior can be specified in the request) and low-level info such as the numbers of tokens used. Please note that this example was directly taken from the OpenAI API Reference [17].

```
1 {  
2     "id": "chatcmpl-abc123",  
3     "object": "chat.completion",  
4     "created": 1677858242,  
5     "model": "gpt-3.5-turbo-0613",  
6     "usage": {  
7         "prompt_tokens": 13,  
8         "completion_tokens": 7,  
9         "total_tokens": 20  
10    },  
11    "choices": [  
12        {  
13            "message": {  
14                "role": "assistant",  
15                "content": "She did not go to the market."  
16            },  
17            "logprobs": null,  
18            "finish_reason": "stop",  
19            "index": 0  
20        }  
21    ]  
22 }
```

Listing 3.6: Sample JSON response from the API.

3.2 RPA Implementation

There are various kinds of actions that the bot performs, but all of them are carried out in the same way I.E., through an UI with buttons, text fields, dropdown lists and so on. Every part that does RPA is therefore made up of these basic blocks, that are shared across the project.

3.2.1 Actions

The exact workflow for performing a task is not described here, but an overview of how these basic challenges have been tackled is hereby provided.

Filling input fields

The first thing you must do to use the application is of course to log in, i.e. to input your access information in a textbox, that is always identified by an ID and can be either an *input* tag for short text like a username or a *textarea* one for long texts like descriptions. To do this, selenium provides a *clear()* and a *send_keys()* method [18], to respectively clear any previously filled text and input a new string.

In the following snippet 3.7 the *textArea* parameter is used to decide which tag to look for using the *find_element()* method [19] with an *XPATH* element locator [20]; the id of the textbox and the content to fill in are of course function parameters.

```
1 from selenium.common.exceptions import NoSuchElementException
2
3 def fill_input(self, id, content, textArea=False):
4     """
5     Fills an input field or text area element with the specified content.
6
7     Args:
8         id (str): The ID of the input field or text area element.
9         content (str): The content to fill the element with.
10        textArea (bool, optional): Whether the element is a text area.
11        Defaults to False.
12
13    Raises:
14        NoSuchElementException: If the element with the specified ID is
15        not found.
16    """
17
18    tagName = 'input'
19    if textArea:
20        tagName = 'textarea'
```

```

21     input_element = self.driver.find_element(By.XPATH, f'//{tagName}[
    @id="{id}"]')
22     input_element.click()
23     input_element.clear()
24     input_element.send_keys(content)
25
26     except NoSuchElementException as e:
27         error_logger.error(f"Error: Element with ID '{id}' not found.
    Cannot fill input.")
28         raise

```

Listing 3.7: Function definition for filling input

Clicking buttons

Buttons are a fundamental part of every application; they are mainly used to confirm or abort actions and they usually have a unique identifier in the DOM. In the applications' HTML, the clickable part of a button is always a link with a unique *arid* attribute.

The following function definition 3.8, that takes the arid and tag type as parameters, uses the *WebDriverWait* [21] feature to wait for the desired element to be clickable [22] (useful for considering page loading); here, an *XPATH* selector is still used to find the element of type *el* and with the arid attribute value equal to *ID*. Finally, the button is pressed using the *click()* [18] method.

```

1 from selenium.common.exceptions import TimeoutException,
    NoSuchElementException
2 from selenium.webdriver.support import expected_conditions as EC
3 from selenium.webdriver.support.ui import WebDriverWait
4
5
6 def pressBtnByARID(self, ID, el="a"):
7     """
8     Presses a button identified by its ARID.
9
10    Args:
11        ID (str): The ARID of the button to press.
12        el (str, optional): The HTML element tag of the button. Defaults
    to "a" (anchor tag).
13
14    Raises:
15        TimeoutException: If the button is not clickable within 15
    seconds.
16        NoSuchElementException: If the button with the specified ARID
    cannot be found.
17    """
18
19    try:

```

```

20         btn = WebDriverWait(self.driver, 15).until(
21             EC.element_to_be_clickable((By.XPATH, f'://{el}[@arid="{ID}"]'
22         ))
23         btn.click()
24
25     except (TimeoutException, NoSuchElementException) as e:
26         if isinstance(e, TimeoutException):
27             error_logger.error(f"Error: Button with ARIA ID '{ID}' not
28             clickable within 15 seconds.")
29         else:
30             error_logger.error(f"Error: Button with ARIA ID '{ID}' not
31             found.")
32         raise

```

Listing 3.8: Function definition for clicking a button

Handling dropdowns

Dropdowns are useful to select from a wide range of values (e.g., which company the user is working for or which role it has), and some lists are even nested two to three times, with every level being a categorization. To fill up a dropdown, its arrow button needs to be clicked to open the list itself (like a regular button) and then, after navigating to the right option it can be finally selected by clicking on it. The interesting part is of course how to navigate a dropdown, and to do this the application provides arrow buttons to scroll up and down the list; clearly, we want to press the down button until we can see the option we want and then click on it. When dealing with nested lists, this process can be repeated as many times as the number of levels.

In the following code snippet 3.9, you can see how the target option that is passed as function parameter is looked for using the *find_element()* method and clicked when found. If it can't be located, the down arrow is pressed (inside the except clause) to advance in the list; this process is repeated up to *retries_threshold* times to make sure that the element lookup does not result in an endless loop. If the element has not been found after the threshold, an error is raised to signal that the option is not present in the dropdown.

```

1 from selenium.common.exceptions import NoSuchElementException,
   TimeoutException
2 from selenium.webdriver.support import expected_conditions as EC
3 from selenium.webdriver.support.ui import WebDriverWait
4
5
6 def selectOption(self, name, retries_threshold=200):
7     """
8     Selects an option from a list by its name.

```

```
9
10     Args:
11         name (str): The name of the option to select.
12         retries_threshold (int, optional): The maximum number of retries
13         before raising an exception. Defaults to 200.
14
15     Raises:
16         NoSuchElementException: If the option with the specified name is
17         not found after retries.
18         TimeoutException: If the down arrow element is not clickable
19         within the wait time.
20     """
21
22     selector = f'//td[text()="{name}"]'
23     down_selector = '//div[@class="MenuOuter"]/div[contains(@class,"
24     MenuScrollDown")]'
25     done = False
26     retries = 0
27
28     while not done:
29         try:
30             retries += 1
31             if retries > retries_threshold:
32                 raise NoSuchElementException(f"Option '{name}' not found
33                 in the list after {retries_threshold} retries.")
34
35             time.sleep(0.1)
36             btn = self.driver.find_element(By.XPATH, selector)
37             btn.click()
38             done = True
39
40         except NoSuchElementException:
41             try:
42                 down_arrow = WebDriverWait(self.driver, 15).until(
43                     EC.element_to_be_clickable((By.XPATH, down_selector))
44                 )
45                 down_arrow.click()
46             except TimeoutException:
47                 error_logger.error("Error: Down arrow element not
48                 clickable within 15 seconds.")
49                 raise
50
51     return None
```

Listing 3.9: Function definition for selecting an option from a dropdown list

Working with several iframes, tabs & windows

Dialogs and modals are a fundamental part of any modern application and they are mainly used to confirm important actions with specific prompts, E.G., asking for a user ID before deleting it from the system and to pick values that need to be searched for, in the former case a search dialog is opened and the selected value is then passed back to the main form. These use cases require a popup behavior, that in this implementation is achieved either by opening a new window or by creating an iframe in the DOM, that acts as a separate page.

The following listing 3.10 demonstrates how a context switch between tabs can be done in Selenium, with a simple method invocation. The function switch to the desired tab index and return the window handle of the previous one. Please note that, from docs [23] *"WebDriver does not make the distinction between windows and tabs. If your site opens a new tab or window, Selenium will let you work with it using a window handle. Each window has a unique identifier which remains persistent in a single session"*.

```
1 from selenium.common.exceptions import NoSuchWindowException
2
3 def switchToNthWindow(self, winNum=1):
4     """
5     Switches the driver's focus to the nth window handle (counting from
6     0).
7
8     Args:
9         winNum (int, optional): The index of the window to switch to.
10        Defaults to 1 (second, freshly opened window).
11
12    Returns:
13        str: The handle of the previously active window, or None if an
14        error occurs.
15
16    Raises:
17        NoSuchWindowException: If the specified window handle is not
18        found.
19        IndexError: If the requested window index is out of range.
20    """
21
22    if winNum < 0:
23        raise IndexError("Window index must be a positive integer.")
24
25    try:
26        all_windows = self.driver.window_handles
27
28        if len(all_windows) < winNum:
29            raise NoSuchWindowException(f"Not enough windows available.
30            Only {len(all_windows)} windows found, requested window #{winNum}")
```

```

26         old_window = self.driver.current_window_handle
27         new_window = all_windows[winNum]
28
29
30         self.driver.switch_to.window(new_window)
31         return old_window
32
33     except (NoSuchWindowException, IndexError) as e:
34         error_logger.error(f"Error switching window: {e}")
35         return None

```

Listing 3.10: Function definition for switching to another tab.

Switching to a nested iframe requires finding the DOM element and passing it to the *switch_to.frame()* function, as here demonstrated 3.11.

```

1 from selenium.common.exceptions import NoSuchElementException
2
3 def switchToNestedIframe(self, target_class):
4     """
5     Switches the driver's focus to a nested iframe within a table cell.
6
7     Args:
8         target_class (str): The class attribute (or part of it) of the
9         table cell containing the iframe.
10
11     Raises:
12         NoSuchElementException: If the target iframe cannot be found.
13     """
14     sel = f'//td[contains(@class,"{target_class}")]/iframe'
15     try:
16         iframe = self.driver.find_element(By.XPATH, sel)
17         self.driver.switch_to.frame(iframe)
18     except NoSuchElementException as e:
19         error_logger.error(f"Error: Nested iframe with class containing '{target_class}' not found: {e}")
20         raise
21
22 def switchBackToDefaultContent(self):
23     """
24     Switches the driver's focus back to the default content frame.
25     """
26     self.driver.switch_to.default_content()

```

Listing 3.11: Function definition for switching to and from a nested iframe.

3.2.2 Error handling

Error handling is perhaps the most delicate topic for any software piece to work effectively, and when not done properly general malfunctioning and specifically crashes can occur.

For example, if the scenario in which an option is not found in a dropdown is not considered, the application will then terminate instead of just aborting the current procedure and moving on. Given this project's short timeframe, not all possible errors have been considered, and the application is not fully resilient. Nevertheless, all the problems encountered during regular use have been handled to enable fault tolerant batch processing.

Logging

State of the art error handling goes together with well-defined logging strategies that require impeccable software architecture. More specifically, the bot keeps records: one for errors and one for general logging. This is done because in the error file, one line corresponds to one problem, and this makes it easy to get a glimpse of how the process is going just by checking its size. The error record includes a brief explanation of the cause, nevertheless it is possible to know more about its dynamic by taking a closer look at the info file, where every action is verbosely logged.

Please note that the *log_to_stdout* parameter present in the following logger configuration method 3.12 makes it possible to log errors to the console and to the log file at the same time, by setting multiple log handlers [24].

```
1 import logging
2
3 def setup_logger(name, log_file, level=logging.INFO, log_to_stdout =
4     False):
5     """
6     Configures and initializes a logger with a file handler for writing log
7     messages.
8
9     Args:
10         name (str): The name for the logger instance.
11         log_file (str): The path to the file where log messages will be
12             stored.
13         level (int, optional): The logging level threshold. Defaults to
14             logging.INFO.
15         log_to_stdout (bool, optional): Flag to enable logging messages to
16             the console as well.
17
18             Defaults to False.
19
20     Returns:
21         logging.Logger: The configured logger object, or None on error.
22
23     Raises:
24         FileNotFoundError: If the specified log file cannot be opened.
25     """
```

```

22 try:
23     file_handler = logging.FileHandler(log_file)
24     formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s
    - %(message)s')
25     file_handler.setFormatter(formatter)
26     logger = logging.getLogger(name)
27     logger.setLevel(level)
28     logger.addHandler(file_handler)
29     if log_to_stdout:
30         stdout_handler = logging.StreamHandler(stream=sys.stdout)
31         logger.addHandler(stdout_handler)
32     return logger
33 except FileNotFoundError as e:
34     print(f"Error: Failed to create logger. Log file '{log_file}' not
    found: {e}")
35     return None

```

Listing 3.12: Function definition for initializing a Logger object.

```

1 error_logger = utils.setup_logger(os.path.join(cwd, "logs", f'ERROR_{time
    .time()}.log'),
2                                     f'ERROR_{time.time()}.log', level=logging.ERROR)
3
4 info_logger = utils.setup_logger(os.path.join(cwd, "logs", f'INFO_{time.
    time()}.log'),
5                                     f'INFO_{time.time()}.log'))
6
7 ...
8
9 error_logger.error(f"User {userID} already had group or relationship {
    Group}")

```

Listing 3.13: Logger usage in the application.

Explicit errors

There are two ways an unintended situation can be noticed: either there is an explicit error message like that pops up after pressing the confirm button or the problem is before E.G., it is impossible to find the desired element in a list.

The former situation can be handled explicitly by checking for an error message every time an action is confirmed and saving it to the error file, as can be seen in the following function definition 3.14. The save button is pressed and then if the *waitForLoading()* function times out meaning the page is stuck, an error popup is looked for and the message is then included in the thrown Exception.

```

1 def pressSaveOrSearchButton(self):
2     """

```

```

3     Presses the "SAVE" button and waits for loading to complete.
4
5     Raises an exception if saving fails, including an informative error
    message.
6
7     Raises:
8         Exception: If an error occurs while saving.
9     """
10    try:
11        self.pressBtnByARID("SAVE")
12        self.waitForLoading()
13    except InfiniteLoading:
14        errorMessage = "None"
15    try:
16        errorField = self.driver.find_element(By.XPATH, "//td[
contains(@class, 'prompttext prompttexterr')]")
17        errorMessage = errorField.text
18    except NoSuchElementException:
19        errorMessage = "Unknown"
20    finally:
21        raise Exception(f"Could not save request! Message: {
errorMessage}")

```

Listing 3.14: Function definition for handling a simple error popup.

Implicit errors

Conversely, when dealing with missing data the error needs to be noticed "implicitly" like in this previous example 3.9, where if the entry is not found after thoroughly scrolling the dropdown list, an exception is raised.

3.2.3 Configuration

To widen the range of possibilities, some settings have been added to the bot, both general ones such as files paths and specific flags to modify the RPA behavior E.G., skipping some steps or performing a different operation.

All the configuration is located in a config.yml file that is structured hierarchically into categories in order to be visually intuitive, as can be seen in the following example 3.15.

```

1 login_data:
2     address: www.example.com
3     user: admin
4     pwd: password123
5 general_options:
6     load_file_name: example.csv
7     putLicense: True

```

```
8 openNewTab: False
```

Listing 3.15: Example of the config.yml file.

Reading the file is then extremely easy with python's YAML library [25] as shown in the following snippet 3.16.

```
1 import yaml
2 def loadConfig(path):
3     """
4     Loads configuration data from a YAML file.
5
6     Args:
7         path (str): The path to the YAML file containing configuration
8         data.
9
10    Returns:
11        dict: The parsed configuration data as a dictionary, or None if
12        an error occurs.
13
14    Raises:
15        FileNotFoundError: If the specified file is not found.
16        IsADirectoryError: If the specified path is a directory.
17        yaml.YAMLError: If an error occurs while parsing the YAML file.
18    """
19    try:
20        with open(path, "r") as ymlfile:
21            return yaml.safe_load(ymlfile)
22    except FileNotFoundError as fnf_error:
23        error_logger.error(f"Error: {fnf_error}")
24    except IsADirectoryError as dir_error:
25        error_logger.error(f"Error: The specified path '{path}' is a
26        directory.")
27    except yaml.YAMLError as yaml_error:
28        error_logger.error(f"Error parsing YAML file: {yaml_error}")
29    except Exception as e:
30        error_logger.error(f"An unexpected error occurred: {e}")
31
32    return None
33
34 ...
35
36 cfg = loadConfig(file_path) # Loading the config parameters
37 if cfg is not None:
38     cfg["general_options"]["openNewTab"] # Accessing a specific voice
```

Listing 3.16: Function definition and example of reading the configuration file.

3.3 Ticket scraping

After automating the solutions to common problems, a monitoring system was realized to track incidents and user requests. This was also done in an RPA fashion by programmatically opening the ticketing system, checking for new requests and adding them to a csv file to be further processed.

3.3.1 Functioning

The dashboard of the ticketing system consists of a list of open issues; to access an entry's details it is necessary to double click it to open a dedicated page with the same kind of components that were previously discussed such as dropdowns, buttons and inputs.

Therefore, the previously described methods are used to navigate the details page and parse the essential information into a specific *Ticket* object.

Iterating on requests

To gather info about all the tickets it is necessary to iterate on the open issues list, that is a standard HTML table with clickable rows. To make this easier in the code, an *IterOnTable* class has been created to effectively iterate on a HTML table with a for syntax where the loop variable holds the current row (*td* object) that can be directly clicked.

The following implementation 3.17 takes the driver object and the desired table ID as input and produces each element in the table with the proper Python iterator syntax [26].

```
1 from selenium.common.exceptions import NoSuchElementException,
   StaleElementReferenceException
2
3
4 class IterOnTable:
5     """
6     This class iterates over the rows of a table identified by its ID.
7
8     It handles potential exceptions like the table not being found or
9     encountering stale element references during iteration.
10    """
11
12    def __init__(self, driver, table_id):
13        self.driver = driver
14        self.table_id = table_id
15        self.index = 0
16
17        try:
18            self.elements = self.driver.find_elements(
```

```

18         By.XPATH, f'//table[@id="{table_id}"]/tbody/tr[@tabindex
19         ="0"]',
20     )
21     except NoSuchElementException:
22         error_logger.error(f"Error: Table with ID '{table_id}' not
23         found")
24         self.elements = []
25
26         self.max = len(self.elements)
27         self.current = self.elements[self.index] if self.elements else
28         None
29
30     def __iter__(self):
31         return self
32
33     def __next__(self):
34         try:
35             self.elements = self.driver.find_elements(
36                 By.XPATH, f'//table[@id="{table_id}"]/tbody/tr[@tabindex
37                 ="0"]',
38             )
39             self.max = len(self.elements)
40
41             if self.index < self.max - 1:
42                 self.current = self.elements[self.index]
43                 self.index += 1
44                 return self.current
45
46             raise StopIteration
47
48         except (NoSuchElementException, StaleElementReferenceException)
49         as e:
50             error_logger.error(f"Error: Unable to access table row: {e}")
51             raise StopIteration

```

Listing 3.17: Class definition for a table iterator.

The following snippet 3.18 illustrates usage of the iterator and demonstrates how a double click is performed.

From docs [27]: *"ActionChains are a way to automate low level interactions such as mouse movements, mouse button actions, key press, and context menu interactions. This is useful for doing more complex actions like hover over and drag and drop".*

```

1 actionChains = ActionChains(driver)
2 for row in IterOnTable(driver, "Tickets Table"):
3     actionChains.double_click(row).perform()
4     scrapeDetails()

```

Listing 3.18: Code snippet for iterating on table.

The double click action opens a new tab for each row that is processed by the *scrapeDetails()* method by navigating it, collecting the useful information and closing it.

3.3.2 Storage

All the Tickets objects are stored in a dictionary with the request id as key, so that checking whether a request is already in the system is a $O(1)$ time operation in average cases, as stated in the docs [28].

Persistence is achieved by saving the ticket dictionary to disk using the *Pickle* serialization module [29] that implements binary protocols for serializing and de-serializing a Python object structure. The binary file is read before scanning the tickets to avoid duplicates and is used by the scraping and parsing modules, the latter of which will be presented in the following chapter. Please note that *pickled* files are not human readable. The following 3.19 code illustrates the process of loading and dumping, carried out respectively at the beginning and end of execution.

```

1 def load(filename):
2     """Loads ticket data from a pickle file.
3
4     Args:
5         filename (str): The path to the pickle file containing ticket
6         data.
7
8     Returns:
9         list: A list of Ticket objects, or None if an error occurs.
10
11     Raises:
12         FileNotFoundError: If the specified file is not found.
13         pickle.PickleError: If an error occurs during unpickling the data
14         .
15     """
16
17     try:
18         with open(filename, "rb") as file:
19             data = pickle.load(file)
20             return data["tickets"]
21     except FileNotFoundError as e:
22         error_logger.error(f"Error: File not found: {filename}")
23         return None
24     except pickle.PickleError as e:
25         error_logger.error(f"Error: Failed to unpickle data from {
26 filename}: {e}")
27         return None
28
29 def dump(filename, tickets):
30     """Saves ticket data to a pickle file.
```

```

28
29     Args:
30         filename (str): The path to the pickle file to save ticket data
to.
31         tickets (list): A list of Ticket objects to be saved.
32     """
33
34     data = {"tickets": tickets}
35     try:
36         with open(filename, "wb") as file:
37             pickle.dump(data, file, protocol=pickle.DEFAULT_PROTOCOL)
38             error_logger.error(f"Ticket data saved successfully to {
filename}")
39     except (IOError, pickle.PickleError) as e:
40         error_logger.error(f"Error: Failed to save ticket data to {
filename}: {e}")

```

Listing 3.19: Functions definitions to dump and load Ticket objects.

A human readable *CSV* file is also produced by the following function 3.20 and later used to perform analytics on the requests using the python csv module [30].

```

1  import csv
2
3  def saveToCSV(tickets, filename_tickets):
4      """Saves ticket data to a CSV file.
5
6      Args:
7          tickets (dict): A dictionary of ticket data, where keys are
ticket IDs and
8                          values are Ticket objects.
9          filename_tickets (str): The path to the CSV file to save ticket
data to.
10
11      Raises:
12          IOError: If an error occurs while opening or writing to the CSV
file.
13      """
14
15      try:
16          with open(filename_tickets, "w", newline='') as csvFile:
17              writer = csv.writer(csvFile, delimiter=';', quotechar='"',
quoting=csv.QUOTE_ALL)
18              writer.writerow(Ticket.getCSVHeader())
19              for ticket_id, ticket in tickets.items():
20                  try:
21                      csv_data = ticket.getCsvFormat()
22                      writer.writerow(csv_data)
23          except IOError as e:

```

```

24 error_logger.error(f"Error: Failed to save ticket data to CSV
    file: {e}")

```

Listing 3.20: Functions definitions for saving CSV file.

3.3.3 State diagram

A ticket object should represent a request inside the system during its lifecycle that includes gathering all the info from the platform, elaborating a solution and ultimately executing it. This is described in the following state diagram 3.1, taking into consideration both decision errors by the model and runtime ones.

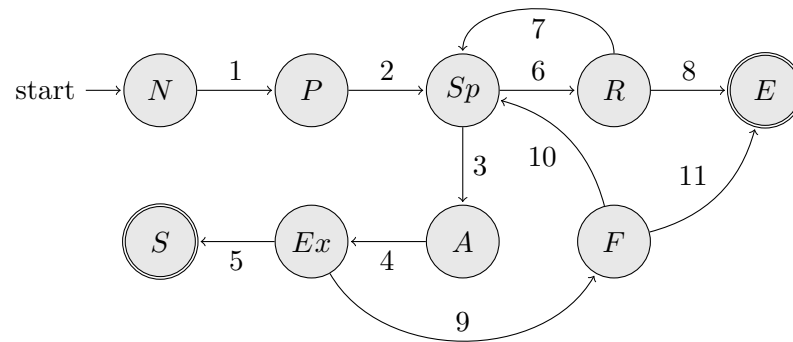


Figure 3.1: State diagram for the *Ticket* class

1	Ticket parsed correctly
2	Solution generated by model
3	Solution approved by user
4	Execution in progress
5	Success
6	Solution rejected by user
9	Execution failed
7-10	Retry generating solution
8-11	Abort ticket

Figure 3.2: Legend for State transitions

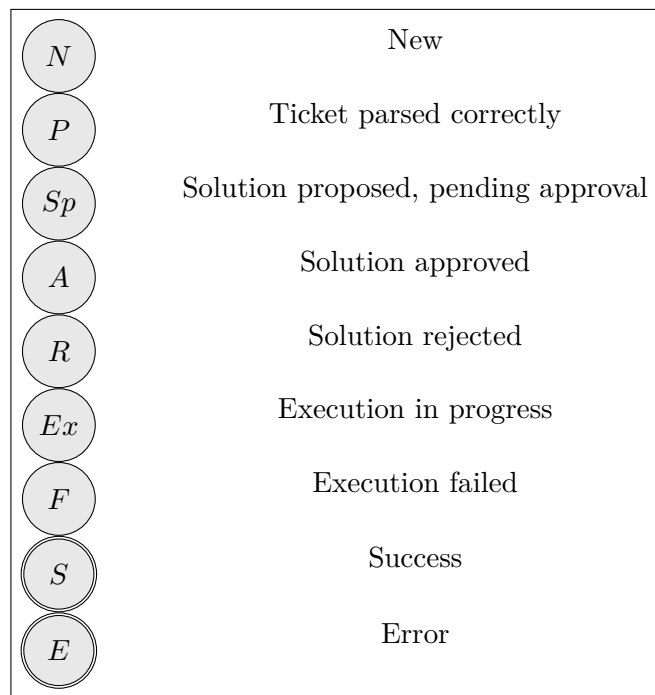


Figure 3.3: Legend for state names

3.4 Request parsing

The data from the scraping is then fed into a simple parser that, using OpenAI's GPT 3.5 model can decide if a problem is solvable using the available automation tools. If that's the case, it proposes a solution (I.E., an invocation of a script) that needs to be reviewed by the user. After processing all the saved requests, the system prompts the user whether they want to run, change or abort the proposed fix and the execution is carried out at once.

3.4.1 Querying GPT

For each saved ticket, the parser prompts the GPT model with the following code and saves the resulting string to a dedicated field in the object.

```

1 def promptModel(query, sys_prompt = SYSTEM_PROMPT):
2     """
3     This function sends a prompt and query to a large language model (LLM)
4     and returns the generated response.
5
6     Args:
7         query: The user's question or text prompt for the LLM.
```

```

8     sys_prompt (optional): A system prompt to guide the LLM
9         (defaults to SYSTEM_PROMPT, a global variable containing
10         instructions or information for the LLM).
11
12     Returns:
13         str: The generated response from the LLM, or an error message
14             if an exception occurs.
15     """
16     try:
17         messages = [
18             {"role": "system", "content": sys_prompt}
19             {"role": "user", "content": query}
20         ]
21         response = openai.ChatCompletion.create(
22             model="gpt-3.5",
23             messages=messages,
24             max_tokens=100,
25             n=1,
26             stop=None,
27             temperature=0.7
28         )
29         return response.choices[0].message['content']
30     except Exception as e:
31         return f"An error occurred: {e}"

```

Listing 3.21: Function definition for querying GPT model.

An optimal system prompt containing specific context information has been created to explain the model precisely which tools it can use and how to invoke them.

3.4.2 Approving a solution

After all the requests have been processed, a user must review the proposed solutions one by one to assess their correctness. This is done with a simple CLI interface that enables them to approve, reject or even change the proposed module invocation.

```

1
2 def promptUser(tickets):
3     """
4     This function iterates through a list of tickets and prompts the user
5     for approval
6     of a proposed solution for each ticket. The user can either approve,
7     reject or edit it and the objects are modified in-place.
8
9     Args:
10         tickets: A list of Ticket objects
11     """
12     for ticket in tickets:
13         print(str(ticket))

```

```

12     inpt = ""
13     while inpt != 'Y' and inpt != 'N' and inpt != 'M':
14         inpt = input("Do you approve the proposed solution? (Yes,No,
Modify): ")
15     if inpt == 'Y':
16         ticket.approveSolution()
17     elif inpt == 'N':
18         ticket.discardSolution()
19     else:
20         solution = None
21         while solution is None:
22             try:
23                 solution = input("Enter your solution: ")
24                 if not solution:
25                     raise ValueError("Solution cannot be empty.")
26                 confirmation = input("Is this correct? (y/n): ").lower()
27                 if confirmation != "y":
28                     solution = None
29             except ValueError as e:
30                 print(f"Error: {e}")
31             ticket.setSolution(solution)

```

Listing 3.22: Function definition for displaying and handling approval prompt.

3.4.3 Executing

Finally, all the requests with positive feedback are solved by launching the specified command. This is done low-level with a subprocess call, that starts a new process with the given CLI command and returns the exit code with the console output log. Clearly, this requires the error logger 3.12 to output all the logs to STDOUT in order to be able to capture it and report back to the ticket object any errors.

```

1 for ticket in tickets:
2     try:
3         if ticket.toExecute():
4             completed_process = subprocess.run(
5                 ticket.getSolution(), capture_output=True, timeout=300,
6                 check=True)
7             ticket.setSuccessState()
8     except (subprocess.CalledProcessError, TimeoutError) as e:
9         if isinstance(e, subprocess.CalledProcessError):
10             err_msg = f"Ticket {ticket.id} solution failed with exit code {e.
returncode}. "
11             if e.stderr:
12                 err_msg += f"Error log: {e.stderr.decode()}."
13             elif isinstance(e, TimeoutError):
14                 err_msg = f"Ticket {ticket.id} solution timed out after 300 seconds
"

```

```
15     ticket.setErrorState(err_msg)
16     error_logger.error(err_msg)
```

Listing 3.23: Code snippet for executing ticket solution.

4

Results

In this chapter, a critical analysis of the data gathered during the day-to-day use is presented.

This information is turned into metrics which are both quantitative and qualitative, offering technical insights like speed and accuracy as well as practical ones i.e., how useful the bot could be in terms of how many problems it can solve.

4.1 Bot Performance

Evaluating the bot quantitatively requires choosing some KPI (Key Performance Indicators) such as speed and accuracy rate. For this matter, three common tasks have been chosen that respectfully require adding, linking and editing objects. Please note that the following metrics are all calculated from the same dataset i.e., executing the same exact tasks.

4.1.1 Execution time

In the first figure 4.1, execution time is shown to be significantly shorter with the bot compared to when a person is doing it; in addition to this, we can see that the bot merely goes below the average time, but it can surpass it significantly in isolated cases. This is due to scrolling being implemented in a slow way, so for the bot to reach the end of a dropdown list can take a long time; in fact, scrolling in general is the single thing that contributes the most to the dispersion of this data, for both parties.

On the other hand, humans often take less time than average when dealing with

“easy” tasks such as when the item they are looking for is at the top of a list or even when they have been performing the same task repetitively.

The data has been gathered by clocking every task from the moment the browser opens to the moment it closes, this was done automatically for the bot. Please note that the error bars show the minimum and maximum value (not the standard deviation), and the sample size differs by one order of magnitude i.e. 10^4 for the bot and 10^3 for manual measurements.

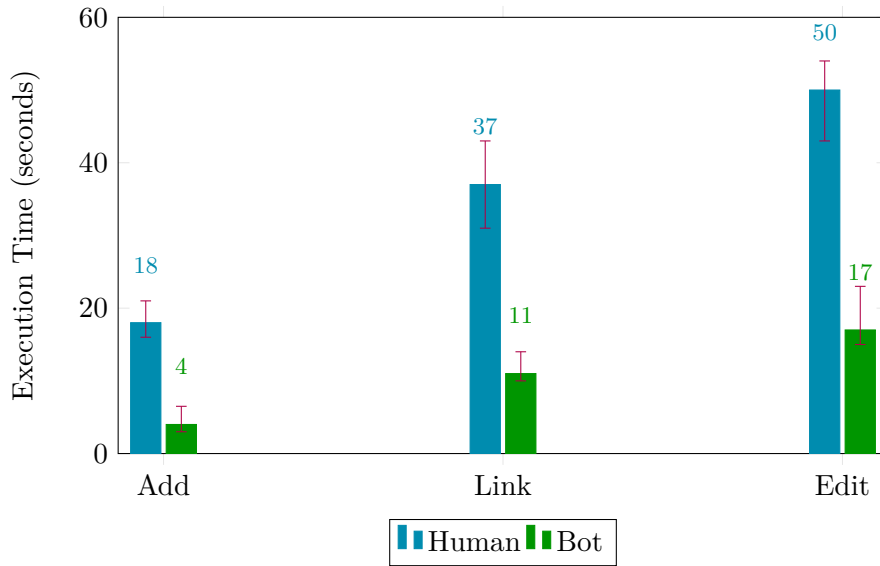


Figure 4.1: Comparison of Execution Times for Three Tasks: Human vs. Bot

4.1.2 Error rate

Here, the frequency of unexpected situations occurring is analyzed and divided into warnings and errors: the *Warnings* category accounts for cases in which the normal bot functioning is not altered, but a task is simply skipped and are usually caused by inconsistent input; *Errors* on the other hand cause the application to crash or behave unexpectedly and are originated by bugs in the code. Please note that the following table 4.1 and bar chart 4.2 display the same information, and that the data is generated by a Python script that processed several log files producing a CSV file.

In the bar chart, green represents Successes, blue Warnings and red Errors.

As can be noticed, Errors are very uncommon, and this is due to the simple nature of the platform the bot works with: it is static (i.e. the website never changes) and most of the error handling has been done properly as described in chapter 3.2.2, making the bot resilient to most of the unexpected situations that can be encountered.

Task	Successes	Warnings	Errors	Total	Error rate
Add	678	30	6	714	0.84%
Link	531	93	17	641	2.65%
Edit	259	21	5	285	1.75%
Total	1468	144	28	1640	1.70%

Table 4.1: Number of Occurred Events for Each Task

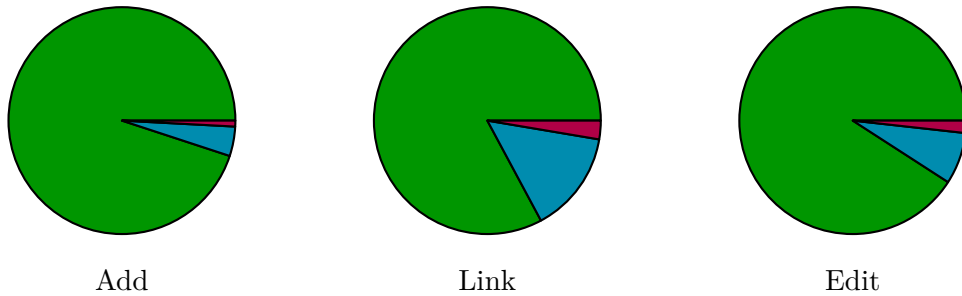


Figure 4.2: Rates of Successes, Errors, and Warnings

Warnings on the other hand are more frequent, especially for the *Link* category and this mostly has to do with inconsistent input values that either result in a search for a non-existent object or cause the application to display an error popup somewhere. Both these situations are logged and notified to the user and would occur even if a human was performing that same task with the same data.

4.2 Qualitative Analysis

The actual usefulness of the bot is now assessed in practical terms i.e., coverage percentage of all encountered requests and time saved.

To do this, all the collected tickets have been individually classified into five categories: three being the implemented tasks (Add, Link, Edit) while the others accounting for invalid and unsupported (Not Handled, N.H.) requests. For the supported categories then, an average saved time per task is computed and then plugged into the total number of processed objects to calculate an estimate of the total saved time.

4.2.1 Request coverage

The following chart 4.3 shows the distribution of the 381 collected tickets.

Not considering the 17 invalid requests, the coverage rate of the system is 254 of 364 (70%) that is overall satisfying.

Raising this would require some work because it requires implementing the missing automation.

Please note that most of the core actions described in chapter 3.2.1 are common to all tasks, so covering a new task type usually just requires using preexistent methods to achieve the desired behavior, and this could arguably be done by a LLM too.

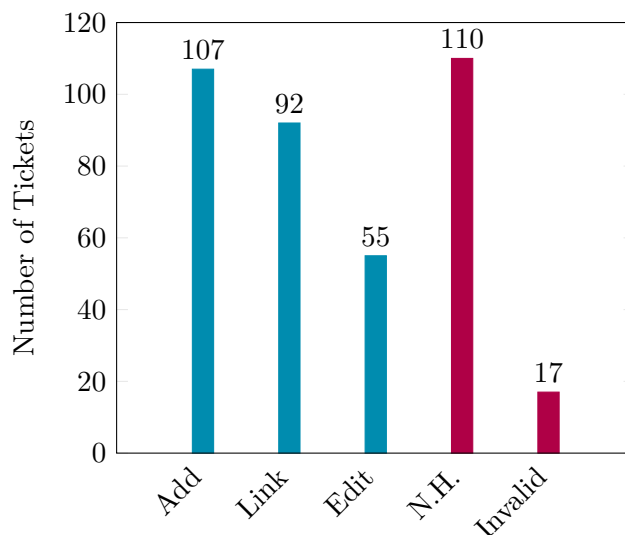


Figure 4.3: Ticket categories

4.2.2 Time saved

Considering the execution time from chapter 4.1 applied to the requests shown in figure 4.3, it would take 8080s (about 2 hours 14 and a half minutes) for a human to complete the 254 tasks, while the bot just took 2375s (about 39 and a half minutes).

This means that, in addition to avoiding the need of human labor, the bot saved 5705s (about 1 and a half hours) for 254 requests, that is 22.5s per task.

Given that a total of 6485 objects have been processed with the bot and considering an average time saving of 22.5s per task, the total saved time is about 40,5h (about five working days) across a project duration of two months and a half.

This number could look exaggerated but a human manually processing 6000 requests would reasonably take between one and two weeks while automated software could do it in two or three working days.

5

Conclusions

In this final chapter, the project's impact and how close it got to its goals are assessed. To do this, achievements, limitations and possible improvements for the project are discussed.

5.1 Key points

The project timeline spanned approximately 2 months and it was carried out by one person. Despite the short timeframe and limited resources, the findings are positive overall.

The early idea, i.e., automating repetitive tasks, was successfully realized. During the first month of development and usage, the software proved effective, useful for batch processing and ultimately saving a considerable amount of time. This initial stage required developing all the core action automation and the error handling mechanisms, which would be fundamental for the subsequent phases.

As can be seen from the metrics in chapter 4, this goal was successfully achieved with an average error rate of 1.7% which could be lowered but can also be already considered a success.

By leveraging AI technologies, the objective then expanded into reducing manual intervention and increasing efficiency and finally turned into creating a system capable of running almost autonomously. This was done by developing a program that would read the incoming user requests automatically and possibly solve them autonomously. Clearly, this part would take more resources to implement because the problem of a system deciding for itself is intrinsically harder than plain scripted automation.

Developing the ticket scraping software and the ticket object lifecycle took less than one week because the core actions were already implemented and resulted in an easy success.

To achieve the requested autonomy, an external intelligent system was used i.e. ChatGPT as a decision-making tool. The chosen model was GPT-3.5, that produced acceptable results after tweaking the system prompt empirically and directly feeding it the requests' text. No metrics are provided for this part because, although it technically worked, insufficient work and testing were conducted.

Surely, what can be said is that AI can be a powerful decision maker that has the potential of enabling a system to interpret and respond to complex service requests autonomously. Even if the project was not fully completed, an important part of automation that's still in use inside ALTEN was realized and has proven to be widely useful.

This led to improved efficiency, enhanced service quality, and ultimately resource optimization. As a result, the team working on this project only needed to manually handle fewer than 20 out of every 1000 requests and could be redirected to more strategic and complex tasks, optimizing overall workforce allocation and reducing the need for manual labor.

5.2 Limitations

Every project, regardless of its successes, encounters certain limitations and challenges. This section discusses the constraints, which understanding is crucial for recognizing areas that require further improvement and setting realistic expectations.

The most obvious limit is the scope of automation being intrinsically confined to the processes defined during the development, without the possibility for the final user to add new workflows. While the bot effectively reduces the workload for certain tasks, handling new ones would require a developer to implement new code, as there is no way for the system to learn by replicating.

Error handling, while implemented, may not cover all potential fault cases. Unexpected errors could interrupt the automation process in isolated cases (less than 2%), requiring human intervention to resolve issues. This underscores the need for more robust testing and error management mechanisms and relates directly to the next point.

Resources constraints were another area of limitation. The project's limited timeframe did not allow for extensive testing and comprehensive evaluations. As a result, the long-term reliability, scalability, and impact of the bot on service efficiency remain partially unexplored. For example, the scalability, bottlenecks and infrastructure/deploy strategy for this system have never been studied.

5.3 Future work and final remarks

As with many pioneering projects, there are many opportunities for future enhancements, both practical and technical.

First, adaptive learning for adding new tasks could be implemented, enabling the user to record his or her actions to "show the bot how it's done". A way could be to inject custom JavaScript into the browser to be able to listen to predefined actions, as WebDriver does not support this kind of recording natively. Another possibility that would require changing the approach completely would be to record a video of the process and make the system learn from it. This can be seen in Large Action/Agentic Models by *"interacting with the real world through integration with external systems [...] LAMs can perform physical actions, control devices, retrieve data, or manipulate information. This permits LAMs to mechanise complete processes and smartly interact with the world, converse with people, adjusting as conditions vary, and even working together with other LAMs."* [31].

An agent, i.e. something that *"perceives its environment, takes actions autonomously in order to achieve goals, and may improve its performance with learning or acquiring knowledge"* [32] could be also used for strengthening the overall decision-making skills of the bot thus reducing the need for human intervention.

This could be realized with tools like LangGraph that enable you to *"build complex, stateful agents without getting bogged down in manual state and interrupt management"* [33] and even implement a *"'divide-and-conquer' approach: create an specialized agent for each task or domain and route tasks to the correct 'expert'"* [4] as it is shown example diagram 5.1. The mechanism of solution proposal and approval could for example be implemented as a conversation between two dedicated agents, resulting in a more structured and autonomous AI system.

For this to be possible, another improvement would be necessary: to conduct systematic, data-driven tuning of the system prompt and model parameters as opposed to empirical tweaking, to achieve optimal performance and accuracy.

In conclusion, this thesis confirmed how automation represents a significant step forward in enhancing efficiency and service quality within service management. By using GPT-3.5 then, the project successfully leveraged the fruits of continued investment and improvements in AI by demonstrating the potential of NLP (Natural Language Processing) for decision making.

This work can be seen as a template for AI-enabled software engineering applied to automation: it provides method and structure while leaving some open questions. This brings naturally to the final takeaway being: as technology continues to evolve, embracing these advancements will be crucial.

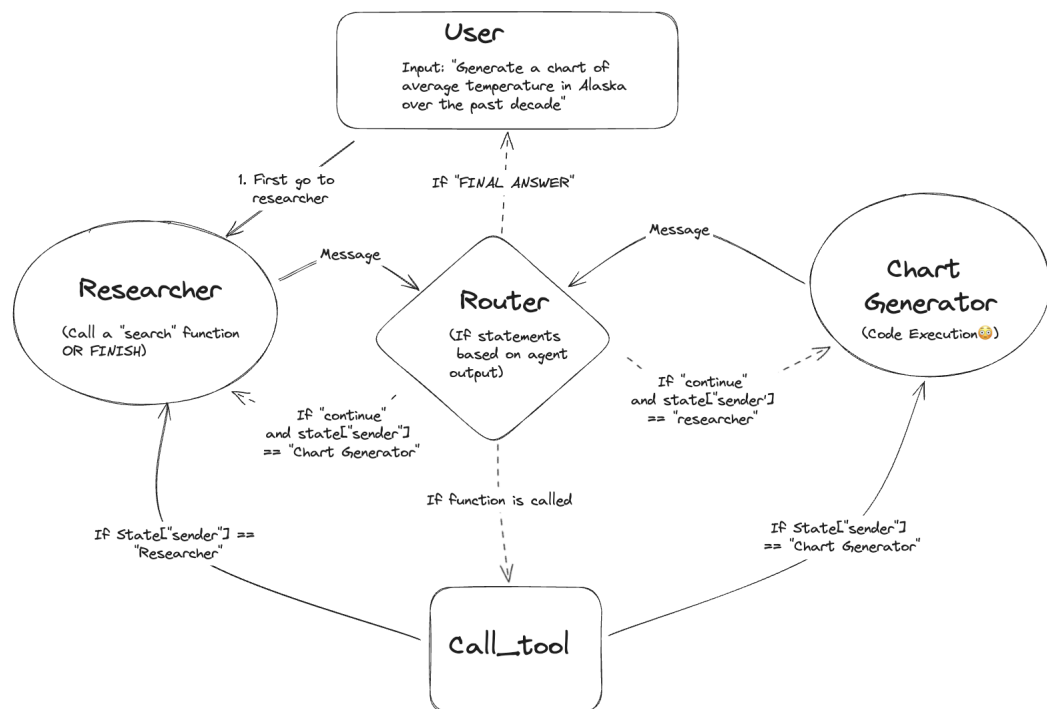


Figure 5.1: Example of multi agent system (from docs) [4]

Bibliography

- [1] I. BMC Software, "bmc helix itsm - bmc software," 2024. <https://docs.bmc.com/docs/servicedesk213/>.
- [2] A. C. Ltd., "rpa overview," 2024. https://www.abeam.com/id/en/service_line/RPA/.
- [3] M. T. KiwiQA, "all you need to know about selenium web-driver architecture," 2022. <https://kiwiqa.co.uk/blog/all-you-need-to-know-about-selenium-webdriver-architecture/>.
- [4] LangGraph, "Basic multi-agent collaboration." https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi-agent-collaboration/, 2024. [Online; accessed 18-June-2024].
- [5] A. Limited, *"ITIL® Foundation, ITIL 4 Edition"*. TSO (The Stationery Office), 2019.
- [6] I. BMC Software, "bmc helix itsm - bmc software," 2024. <https://www.bmc.com/it-solutions/bmc-helix-itsm.html/>.
- [7] A. Group, "leader in engineering and it services," 2024. <https://www.alten.com/>.
- [8] Wikipedia, "robotic process automation," 2024. https://en.wikipedia.org/wiki/Robotic_process_automation/.
- [9] U. Inc., "ai at work-the business automation platform™-leader in rpa, ai & automation | uipath," 2024. <https://www.uipath.com/>.
- [10] S. GitHub, "selenium," 2024. <https://github.com/SeleniumHQ/selenium/>.
- [11] T. W. W. W. C. (W3C), "webdriver specification," 2024. <https://w3c.github.io/webdriver/>.
- [12] S. F. Conservancy, "webdriver | selenium," 2024. <https://www.selenium.dev/documentation/webdriver/>.

-
- [13] Wikipedia contributors, "Openai — Wikipedia, the free encyclopedia." <https://en.wikipedia.org/w/index.php?title=OpenAI&oldid=1225767408>, 2024. [Online; accessed 27-May-2024].
 - [14] I. OpenAI, "openai api," 2024. <https://openai.com/index/openai-api/>.
 - [15] P. S. Foundation, "download python | python.org," 2024. <https://www.python.org/downloads/>.
 - [16] I. OpenAI, "openai python api github," 2024. <https://github.com/openai/openai-python/>.
 - [17] I. OpenAI, "openai api examples," 2024. <https://platform.openai.com/examples/>.
 - [18] Selenium, "interacting with web elements," 2024. <https://www.selenium.dev/documentation/webdriver/elements/interactions/>.
 - [19] Selenium, "interacting with web elements," 2024. <https://www.selenium.dev/documentation/webdriver/elements/finders/>.
 - [20] Selenium, "interacting with web elements," 2024. <https://www.selenium.dev/documentation/webdriver/elements/information/>.
 - [21] Selenium, "waiting strategies," 2024. <https://www.selenium.dev/documentation/webdriver/waits/>.
 - [22] Selenium, "waiting with expected conditions," 2024. https://www.selenium.dev/documentation/webdriver/support_features/expected_conditions/.
 - [23] Selenium, "working with windows and tabs," 2024. <https://www.selenium.dev/documentation/webdriver/interactions/windows/>.
 - [24] P. S. Foundation, "logging handlers," 2024. <https://docs.python.org/3/library/logging.handlers.html>.
 - [25] P. team, "pyyaml docs," 2024. <https://pyyaml.org/wiki/PyYAMLDocumentation/>.
 - [26] P. S. Foundation, "iterator - python wiki," 2024. <https://wiki.python.org/moin/Iterator/>.
 - [27] Selenium, "action chains docs," 2024. https://www.selenium.dev/selenium/docs/api/py/webdriver/selenium.webdriver.common.action_chains.html.

-
- [28] P. S. Foundation, “"time complexity",” 2024. <https://wiki.python.org/moin/TimeComplexity/>.
- [29] P. S. Foundation, “"pickle - python object serialization",” 2024. <https://docs.python.org/3/library/pickle.html>.
- [30] P. S. Foundation, “"csv file reading and writing",” 2024. <https://docs.python.org/3/library/csv.html>.
- [31] Rosemary J Thomas, PhD, “The rise of large action models, lams: How ai can understand and execute human intentions?.” <https://medium.com/version-1/the-rise-of-large-action-models-lams-how-ai-can-understand-and-execute-human-intentions> 2024. [Online; accessed 18-Jun-2024].
- [32] Wikipedia contributors, “Intelligent agent — Wikipedia, the free encyclopedia.” https://en.wikipedia.org/w/index.php?title=Intelligent_agent&oldid=1223026734, 2024. [Online; accessed 18-June-2024].
- [33] LangGraph, “Langgraph - langgraph.” <https://langchain-ai.github.io/langgraph/#overview>, 2024. [Online; accessed 18-June-2024].